Firebase Push Notifications

Complete implementation guide for Flutter (Android and iOS)
Setup, architecture, multi-device login, token management, edge cases and pricing

Item	Value
App	`push_notification_demo` (Flutter, go_router)
Reusable package	`packages/fcm_push_kit` v0.1.0
Plugins	`firebase_core ^5.1.0`, `firebase_messaging ^17.0.1`, `flutter_local_notifications ^22.3.3`
Firebase project	`push-notification-bfe0b`
Platforms	Android (API 21+, runtime permission on 13+), iOS (APNs)
Document date	4 November 2026

How to read this document

Sections 1 to 6 describe what is built and working in this repository today. Sections 7 to 10 describe the server side and the account (login) layer that a production app needs on top of it: the client package deliberately has no backend, so those parts are given as a recommended design with sample code. Section 11 covers pricing and section 12 is the go-live checklist. Sections 13 to 15 cover the hybrid topic and backend model, notification action buttons and multi-screen navigation; these are designs with sample code and are not yet in the demo app.

Contents

1. How Firebase Cloud Messaging works
2. Firebase project setup
3. Android integration
4. iOS integration
5. Flutter integration with fcm_push_kit
6. Message types, app states and deep links
7. Recommended architecture
8. Token management
9. Multiple-device and multiple-account login
10. Edge cases and how to handle them
11. FCM pricing, quotas and cost estimates
12. Testing, OS versions, test scenarios and go-live checklist
13. Hybrid delivery model: topics and backend-managed sends
14. Notification action buttons
15. Multi-screen navigation from one tap

Appendix A. FCM error codes

Appendix B. Limits reference

1. How Firebase Cloud Messaging works

Firebase Cloud Messaging (FCM) is Google's free message relay. Your server never talks to phones directly: it sends a message to FCM, and FCM delivers it through the platform push channel. On Android that is Google Play services, and on iOS FCM forwards the message to Apple Push Notification service (APNs).



Key terms

Term	Meaning
Registration token	Opaque string that identifies one app install on one device. It is the address you send to. It changes on reinstall, data clear, restore, and periodically.
APNs token	Apple's device token. FCM needs it before it can issue an FCM token on iOS. The package waits up to 5 seconds for it (`_waitForApnsToken`).
Topic	Named group (e.g. `all`, `offers`). Devices subscribe; the server sends one message to the topic and FCM fans it out.
Notification message	Has a `notification` block (title, body). The OS shows it automatically when the app is in the background or terminated.
Data message	Only a `data` block. Delivered to app code; nothing is shown unless the app shows it.
HTTP v1 API	The only supported send API. The legacy server-key API was shut down in 2024. Authentication uses a service account (OAuth 2.0), not a server key.
FID	Firebase Installation ID. `firebase_messaging` 17 adds FID-based registration (`register()`, `onRegistered`). The package exposes it but still uses FCM tokens, which the console and HTTP v1 `token` target use.

2. Firebase project setup

1. Create a project at console.firebase.google.com (this repo uses `push-notification-bfe0b`). Google Analytics is optional for FCM.
2. Install the CLIs: `npm install -g firebase-tools`, then `dart pub global activate flutterfire_cli`. Run `firebase login`.
3. From the Flutter project root run `flutterfire configure`. It registers the Android and iOS apps, downloads `google-services.json` and `GoogleService-Info.plist`, and generates `lib/firebase_options.dart`.
4. Check that the Android package name and iOS bundle ID registered in Firebase match the `applicationId` in `android/app/build.gradle.kts` and the bundle ID in Xcode. A mismatch is the most common reason a token is issued but nothing arrives.
5. For iOS, upload an APNs Authentication Key (section 4) under Project settings → Cloud Messaging → Apple app configuration.
6. For a backend, create a service account key under Project settings → Service accounts, or better, run the backend on Google Cloud with a service account identity so no key file exists. Never ship this key in the app.

Security: `google-services.json`, `GoogleService-Info.plist` and `firebase_options.dart` contain client identifiers, not secrets; they are safe to commit. The service account JSON is a secret and must never be committed or bundled into the app.

One project per app, or one project for several apps?

Each Firebase project has its own sending quota (600,000 messages per minute by default). Use one project per product so that a large campaign in one app cannot exhaust the quota of another, and so access, analytics and billing stay separate. Use one project with several registered apps only for variants of the same product (e.g. a customer app and a driver app that share users).

3. Android integration

3.1 Gradle

The Google services plugin is declared in `android/settings.gradle.kts` and applied in `android/app/build.gradle.kts`:

```
// android/settings.gradle.kts
plugins {
    id("com.google.gms.google-services") version("4.4.4") apply false
}

// android/app/build.gradle.kts
plugins {
    id("com.android.application")
    id("kotlin-android")
    id("com.google.gms.google-services")
    id("dev.flutter.flutter-gradle-plugin")
}
android {
    compileOptions { isCoreLibraryDesugaringEnabled = true }
}
dependencies {
    // Required by flutter_local_notifications
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")
}
```

3.2 AndroidManifest.xml

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.VIBRATE" />

<application ...>
  <!-- Must equal FcmPushKitConfig.androidChannelId -->
  <meta-data
    android:name="com.google.firebase.messaging.default_notification_channel_id"
    android:value="high_importance_channel" />
  <meta-data
    android:name="com.google.firebase.messaging.default_notification_icon"
    android:resource="@mipmap/ic_launcher" />
  <activity android:name=".MainActivity" android:launchMode="singleTop" ...>
```

3.3 Android behaviour to know

- **Android 13+ (API 33)** requires the runtime `POST_NOTIFICATIONS` permission. `requestPermission()` shows the prompt. If the user denies it twice, Android stops showing the prompt and the user must enable notifications in Settings.

- **Notification channel**: the package creates `high_importance_channel` with `Importance.max`. After creation only the user can change its importance; changing it in code has no effect. To change behaviour, create a channel with a new ID.
- **Small icon**: `@mipmap/ic_launcher` works but shows as a grey square on many devices. For production, add a white-on-transparent drawable (e.g. `@drawable/ic_notification`) and set both the manifest meta-data and `androidSmallIcon`.
- **Foreground**: Android does not show notification messages while the app is open. The package shows them itself through `flutter_local_notifications`.
- **Devices without Google Play services** (Huawei since 2019, some Chinese-market phones, Amazon Fire) cannot receive FCM. They need HMS Push Kit or another provider.

4. iOS integration

4.1 Apple Developer and Firebase

1. In the Apple Developer portal, enable the **Push Notifications** capability on the App ID. A paid Apple Developer Program membership is required.
2. Create an **APNs Authentication Key** (.p8) under Certificates, Identifiers and Profiles → Keys. One key works for all apps in the team and for both sandbox and production, and it does not expire. Prefer it over .p12 certificates, which expire yearly.
3. Upload the .p8 with its Key ID and Team ID in Firebase → Project settings → Cloud Messaging → Apple app configuration.

4.2 Xcode

- Signing and Capabilities: add **Push Notifications** and **Background Modes** with **Remote notifications** (and Background fetch, already set in `Info.plist`).
- `ios/Runner/Runner.entitlements` contains `aps-environment = development`. Xcode switches it to `production` when you archive for App Store or TestFlight. A development build only receives sandbox pushes, and the FCM token is tied to that environment.
- `GoogleService-Info.plist` must be in the Runner target.

```
// ios/Runner/AppDelegate.swift
override func application(_ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
  // Lets firebase_messaging and flutter_local_notifications present
  // notifications and receive taps.
  UNUserNotificationCenter.current().delegate = self as? UNUserNotificationCenterDelegate
  return super.application(application, didFinishLaunchingWithOptions: launchOptions)
}
```

4.3 iOS behaviour to know

- An FCM token is only available after APNs has issued a device token. On a simulator or without network, `getToken()` returns null. Always test push on a real iPhone.
- The permission prompt is shown once. After a denial, only Settings can re-enable it. Show your own explanation screen first, then call `requestPermission()` (set `requestPermissionOnInit: false` for this).
- iOS shows foreground notifications natively via `setForegroundNotificationPresentationOptions`, so the package does not show a local copy on iOS. This avoids duplicate banners.
- If the user swipes the app away from the app switcher, iOS does not deliver data-only (silent) messages until the user opens the app again. Notification messages still show.

- Silent pushes (`content-available: 1`) are throttled by iOS and can be delayed or dropped. Do not rely on them for anything important.
- To show images or change content before display, add a Notification Service Extension. It is not needed for title and body.

5. Flutter integration with fcm_push_kit

5.1 Package structure

File	Responsibility
`lib/src/fcm_push_kit.dart`	Singleton `FcmPushKit.instance`: initialization, permission, token, topics, foreground display, tap streams.
`lib/src/fcm_push_kit_config.dart`	`FcmPushKitConfig`: channel, icon, route key, topics, background handler, logging. Every field has a default.
`lib/src/push_notification_tap.dart`	`PushNotificationTap` and `TapSource` (foreground, background, terminated); builds a go_router location from the data payload.

5.2 Initialization (lib/main.dart)

```
@pragma('vm:entry-point')
Future<void> backgroundMessageHandler(RemoteMessage message) async {
  await fcmPushKitDefaultBackgroundHandler(message);   // initializes Firebase
  debugPrint('Background message: ${message.messageId} ${message.data}');
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await FcmPushKit.instance.initialize(
    firebaseOptions: DefaultFirebaseOptions.currentPlatform,
    config: const FcmPushKitConfig(
      topics: ['all'],
      backgroundHandler: backgroundMessageHandler,
      enableLogs: true,
    ),
  );
  AppRouter.setupNotificationDeepLinkListener();
  runApp(const MyApp());
}
```

What `initialize()` does, in order: initializes Firebase if needed; registers the background handler; initializes local notifications and creates the Android channel; sets iOS foreground presentation; listens to `onMessage`, `onMessageOpenedApp` and `onTokenRefresh`; requests permission (unless disabled); subscribes to the configured topics. It is safe to call twice.

5.3 Public API

Member	Use
`initialize({firebaseOptions, config})`	Once in main() before runApp.
`requestPermission()`	Shows the OS prompt; returns true if authorized or provisional.
`getPermissionStatus()`	Reads the current status without prompting. The home screen re-reads it on app resume to catch changes made in Settings.
`getToken()`	Current FCM token or null. Waits for APNs on iOS.
`onTokenRefresh`	Stream of new tokens. **Send each one to your backend.**
`deleteToken()`	Invalidates this install's token (use on logout).
`subscribeToTopic / unsubscribeFromTopic`	Topic membership for this install.
`onForegroundMessage`	Messages received while the app is open.
`onNotificationTap`	Taps while the app is running (foreground or background).
`getInitialTap()`	The tap that launched the app from terminated state. Returns a value at most once; call it after the first frame.
`showNotification(...)`	Local notification with no server; tap is delivered like a push tap.
`register / unregister / onRegistered`	FID-based registration (optional, for future migration).

5.4 Adding the package to another app

```
dependencies:
  firebase_core: ^5.1.0
  fcm_push_kit:
    git:
      url: https://github.com/Soumya8792/push_notification.git
      path: packages/fcm_push_kit
      ref: fcm_push_kit-v0.1.0
```

Then run `flutterfire configure` for that app's Firebase project and repeat sections 3 and 4.

6. Message types, app states and deep links

6.1 What happens in each state

App state	Notification message	Data-only message	User taps
Foreground	onMessage fires. Android: package shows a local notification. iOS: system banner via presentation options.	onMessage fires. Nothing is shown unless you show it.	Android: local-notification callback → `TapSource.foreground`. iOS: `onMessageOpenedApp`.
Background	OS shows it. Dart onMessage does not fire.	Background handler runs in a separate isolate.	`onMessageOpenedApp` → `TapSource.background`.
Terminated	OS shows it.	Android: background handler runs. iOS: not delivered if the user force-quit.	`getInitialMessage()` via `getInitialTap()` → `TapSource.terminated`.

6.2 Background handler rules

- Must be a top-level or static function with `@pragma('vm:entry-point')`, or release builds tree-shake it away.
- Runs in its own isolate: no access to app state, providers or the router. Initialize Firebase first (the default handler does this).
- Keep it short. Android allows about 20 seconds; long work should be scheduled with WorkManager.

6.3 Deep link payload

Put the route in the `data` block under the key `route` (configurable with `routeKey`). All other data keys become query parameters.

```
{
  "message": {
    "token": "<FCM token>",
    "notification": { "title": "Order shipped", "body": "Tap to track" },
    "data": { "route": "/details", "id": "42", "source": "order" },
    "android": { "priority": "HIGH",
                 "notification": { "channel_id": "high_importance_channel" } },
    "apns": { "payload": { "aps": { "sound": "default" } } }
  }
}
// PushNotificationTap.location == "/details?id=42&source=order"
```

- All `data` values must be strings. Send `"42"`, not `42`.
- Unknown routes fall back to the home screen through `errorBuilder` in `lib/router/app_router.dart`.
- Never trust payload data for authorization. A route like `/orders?id=42` must still check on the server that the signed-in user owns order 42.

6.4 Authentication, invalid and expired deep links

The demo router has no login, so every route is public and unknown routes fall back to home. A production app adds a single go_router `redirect` that runs for every navigation, including notification taps. Nothing in `fcm_push_kit` changes.

```
// lib/router/app_router.dart (production addition)
const _publicRoutes = {'/', '/login'};
const _knownRoutes = {'/', '/login', '/details', '/profile'};

redirect: (context, state) {
  final path = state.uri.path;
  final params = state.uri.queryParameters;

  // 1. Unknown route in the payload: go home with a message.
  if (!_knownRoutes.contains(path)) return '/?error=unknown_link';

  // 2. Expired link: payload carries "expires_at" (epoch seconds, as a string).
  final exp = int.tryParse(params['expires_at'] ?? '');
  if (exp != null &&
      DateTime.now().millisecondsSinceEpoch ~/ 1000 > exp) {
    return '/?error=expired_link';
  }

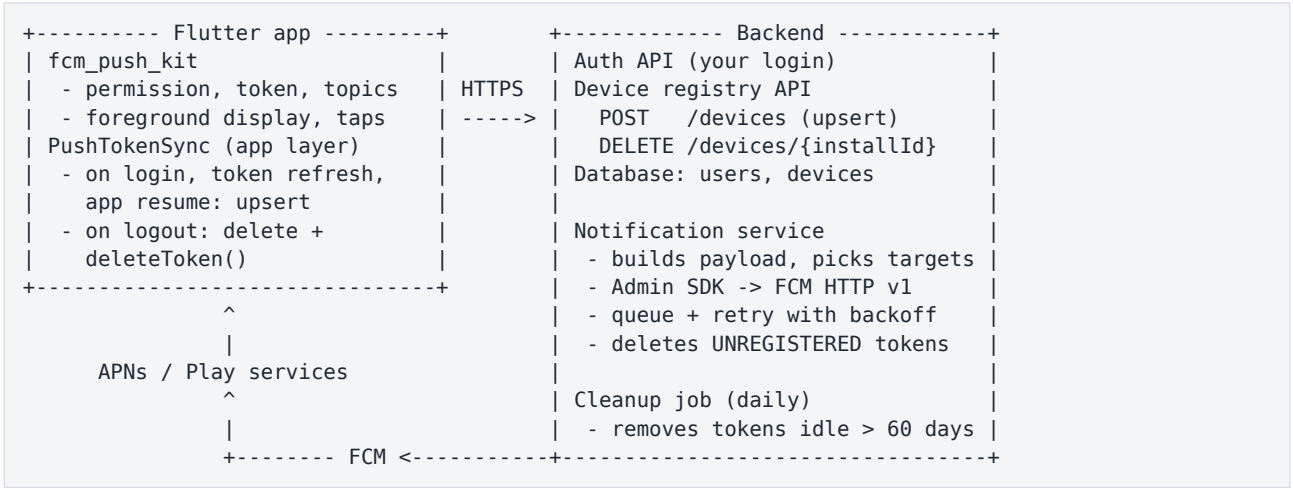
  // 3. Wrong account: payload carries "account_id".
  final target = params['account_id'];
  if (target != null && auth.isSignedIn && target != auth.userId) {
    return '/?error=other_account';
  }

  // 4. Not signed in: send to login and remember where to go next.
  if (!auth.isSignedIn && !_publicRoutes.contains(path)) {
    return '/login?next=${Uri.encodeComponent(state.uri.toString())}';
  }
  return null; // allowed
},
refreshListenable: auth, // re-runs redirect after login/logout
```

Scenario	Expected behaviour
Valid route, signed in	Opens the target screen with its parameters.
Valid route, signed out	Login screen; after sign-in, `next` is opened once and then cleared.
Unknown route (`/abc`)	Home screen with a short "link not available" message. No crash, no blank screen.
Missing `route` key	App opens normally (last screen or home). Tap is logged.
Expired (`expires_at` in the past)	Home with "this offer has ended". The server should also stop sending after expiry by setting `ttl`.
Item deleted or no access	Screen loads, API returns 404 or 403, and the screen shows an empty state with a back button. Never trust the payload for access.
Malformed parameter (`id=abc`)	Screen validates and shows an error state; it must not throw on parse.
Payload for another account	Ignored or shows "sign in as that account to view".

7. Recommended architecture

The client package handles everything on the device. A production system adds three pieces around it: a device registry, a send service and a cleanup job.



7.1 Design principles

- **Address users, not devices.** Product code calls `notifyUser(userId, payload)`. The notification service looks up that user's active devices. Callers never handle tokens.
- **One row per app install**, keyed by a stable installation ID generated on the device (a UUID stored in secure storage), not by the token. Tokens change; the install does not. This makes token refresh an update instead of a new row.
- **Queue sends.** Put send jobs on a queue (Cloud Tasks, Pub/Sub, SQS, BullMQ). Workers call FCM and retry 429/5xx with exponential backoff. API requests never wait on FCM.
- **Use topics for broadcasts** (all users, marketing segments) and tokens for personal messages (orders, chat, security alerts). One topic send costs one API call regardless of audience size.
- **Log the FCM message ID** returned by each send, with user ID and campaign ID, so delivery questions can be traced with FCM delivery data in the Firebase console and BigQuery export.

7.2 Device registry schema

```
-- SQL example; the same shape works in Firestore (users/{uid}/devices/{installId})
CREATE TABLE devices (
  install_id     UUID PRIMARY KEY,            -- generated on device, survives token refresh
  user_id        UUID NULL REFERENCES users,  -- NULL when logged out
  app_id         TEXT NOT NULL,               -- which of your apps (multi-app setups)
  platform       TEXT NOT NULL,               -- 'android' | 'ios'
  fcm_token      TEXT NOT NULL UNIQUE,        -- unique: one token belongs to one install
  permission     TEXT NOT NULL,               -- authorized | denied | provisional
  app_version    TEXT,
  locale         TEXT,
  last_seen_at   TIMESTAMPTZ NOT NULL,        -- updated on every app open
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON devices (user_id) WHERE user_id IS NOT NULL;
CREATE INDEX ON devices (last_seen_at);
```

7.3 Sending with the Admin SDK (Node.js)

```
import { initializeApp } from 'firebase-admin/app';
import { getMessaging } from 'firebase-admin/messaging';
initializeApp();   // uses GOOGLE_APPLICATION_CREDENTIALS or the Cloud runtime identity

const DEAD = new Set(['messaging/registration-token-not-registered',
                      'messaging/invalid-registration-token']);

export async function notifyUser(userId, { title, body, data }) {
  const devices = await db.devices.findMany({ where: { userId, permission: { not: 'denied' } } });
  if (devices.length === 0) return;
  const res = await getMessaging().sendEachForMulticast({   // up to 500 tokens per call
    tokens: devices.map(d => d.fcmToken),
    notification: { title, body },
    data,                                                    // string values only
    android: { priority: 'high', notification: { channelId: 'high_importance_channel' } },
    apns: { payload: { aps: { sound: 'default' } } },
  });
  res.responses.forEach((r, i) => {
    if (!r.success && DEAD.has(r.error.code)) {
      db.devices.delete({ where: { installId: devices[i].installId } });
    }
  });
}

// Broadcast: one call, FCM fans out
await getMessaging().send({ topic: 'all', notification: { title, body } });
```

8. Token management

8.1 When a token changes

Event	Effect on token	What to do
First launch	New token issued	Upsert to backend.
App reinstalled	New token; old one becomes UNREGISTERED	New install ID too. The old row is removed by send errors or the cleanup job.
App data cleared (Android)	New token	Same as reinstall.
Restore to a new phone	New token; backup may carry the old install ID	The UNIQUE token constraint plus upsert handles it. Exclude the install ID from cloud backup if you need strict per-device identity.
Periodic rotation by FCM	New token via `onTokenRefresh`	Update the row for this install ID.
`deleteToken()`	Old token invalid; next `getToken()` creates a new one	Used on logout.
Inactive 270 days (Android)	FCM expires it	Sends return UNREGISTERED; delete the row.

8.2 Client-side sync (recommended addition)

Today `onTokenRefresh` in the package only logs the token, and the demo shows it on screen for testing. A production app adds a small sync service in the app layer:

```
class PushTokenSync {
  PushTokenSync(this._api, this._installId);
  final DeviceApi _api;
  final String _installId;           // UUID stored in flutter_secure_storage
  StreamSubscription<String>? _sub;

  /// Call after login and on every app resume.
  Future<void> start(String userId) async {
    _sub ??= FcmPushKit.instance.onTokenRefresh.listen((t) => _upsert(userId, t));
    final token = await FcmPushKit.instance.getToken();
    if (token != null) await _upsert(userId, token);
  }

  /// Call on logout, BEFORE clearing the auth session.
  Future<void> stop() async {
    await _sub?.cancel();
    _sub = null;
    try { await _api.deleteDevice(_installId); } catch (_) { /* retried next launch */ }
    await FcmPushKit.instance.deleteToken();
  }

  Future<void> _upsert(String userId, String token) async {
    final status = await FcmPushKit.instance.getPermissionStatus();
    await _api.upsertDevice(installId: _installId, userId: userId, token: token,
        platform: Platform.operatingSystem, permission: status.name);
  }
}
```

8.3 Server-side rules

- **Upsert by install ID.** If another row already holds the same token, reassign it to this install and user (the UNIQUE constraint makes this explicit). This is what makes account switching on a shared phone safe.
- **Delete on hard errors:** `UNREGISTERED` (404) and `INVALID_ARGUMENT` for the token field. Do not delete on 429, 500 or 503; retry those.
- **Refresh** `last_seen_at` **on every app open** (a cheap call from `start()`). Firebase recommends refreshing tokens at least monthly.
- **Daily cleanup:** drop or exclude devices with `last_seen_at` older than about 60 days. Stale tokens waste quota and distort delivery metrics.
- **Respect permission:** store the permission status reported by the app. Sends to a device with notifications disabled succeed at FCM but are never shown.

9. Multiple-device and multiple-account login

FCM has no concept of users; it only knows app installs. Every login scenario is solved in the device registry by moving the `user_id` on a device row.

Scenario	Expected behaviour	How it is achieved
Same user on phone and tablet	Both devices receive personal notifications.	Two device rows with the same `user_id`; `notifyUser` multicasts to both.
Same user on Android and iOS	Both receive.	Same as above; FCM handles the platform difference.
User logs out	That device stops receiving personal notifications; broadcast topics may continue.	`PushTokenSync.stop()`: delete device row, then `deleteToken()`. Unsubscribe from user-specific topics.

Scenario	Expected behaviour	How it is achieved
User B logs in on User A's phone	Only B's notifications arrive; none of A's.	A's logout cleared the row. If A's logout failed (offline, crash), B's upsert reassigns the same install and token to B.
App killed without logout, user switches account	No leak of the previous user's messages.	Upsert on login always overwrites `user_id` for this install.
Logout while offline	No further personal messages after reconnect.	`deleteToken()` still runs locally once online; keep a "pending logout" flag and retry `DELETE /devices` on next launch. Server-side: sends to the deleted token return UNREGISTERED and the row is removed.
Single-session policy (only one device allowed)	Older device is signed out when a new one logs in.	On login, the server sends a data message `{"type":"session_revoked"}` to the user's other devices, then deletes their rows and revokes their sessions. The app clears its session when it receives it.
"Log out of all devices"	Every device stops receiving.	Delete all rows for the user and revoke refresh tokens. Devices clean up locally on next launch when their API calls fail with 401.
Multiple accounts signed in at once (account switcher)	Notifications for every signed-in account, labelled.	Store a list of user IDs per device row (or one row per install and user). Include `account_id` in `data` and switch account before navigating on tap.
Same user, several of your apps	Each app gets only its own notifications.	Rows carry `app_id`. Each app's token belongs to its own Firebase project, so the right project credentials must be used when sending.

Topic caution: topic subscriptions belong to the install, not to the user, and FCM cannot list them. If you use user-specific topics (e.g. `user_123`), you must unsubscribe on logout, and that only works while the old token is still valid. Prefer token sends for anything personal and keep topics for public broadcasts.

10. Edge cases and how to handle them

10.1 Permission and token

Case	Handling
User denies permission	App keeps working. Show an in-app banner with a button to Settings. The home screen re-checks status on resume.
Android 12 or older	Notifications are allowed by default; the prompt is not shown.
iOS provisional authorization	Treated as granted. Notifications go quietly to Notification Center.
`getToken()` returns null	No network, iOS simulator, APNs not ready, or Play services missing. Retry on next resume; `onTokenRefresh` delivers it later.
Token issued before login	Register the device anonymously (`user_id` NULL) to receive broadcasts, then attach the user on login.
Wrong APNs environment	Development builds need the sandbox. TestFlight and App Store use production. The .p8 key covers both; a .p12 certificate covers one.
APNs key missing or revoked in Firebase	Sends fail with `THIRD_PARTY_AUTH_ERROR`. Alert on this error; it affects every iOS user.

10.2 Delivery

Case	Handling
Device offline	FCM stores messages up to the TTL (default and maximum 28 days). Set a short `ttl` for time-sensitive messages such as OTPs.
More than 100 pending messages for an offline device	FCM discards them all and the app gets `onDeletedMessages` (Android). Use a `collapse_key` for "latest state" updates so only the newest one is kept.
Android Doze / battery saver	Use `priority: HIGH` only for messages the user sees. Android may downgrade high priority if the app never shows a notification.
OEM battery killers (Xiaomi, Oppo, Vivo, OnePlus)	Background delivery can be delayed or blocked. Guide affected users to disable battery optimisation for the app.
Android "Force stop"	No messages until the user opens the app again (OS rule).
iOS app swiped away	Notification messages arrive; data-only messages do not.
Duplicate notifications	Can happen when a device is in a topic and also receives a token send, or on retry after a timeout. Send each audience one way, and use the Android `tag` / iOS `apns-collapse-id` to replace instead of stack.
Payload over 4 KB	Rejected with `INVALID_ARGUMENT`. Send an ID and fetch the details from your API.
Per-device rate	Over about 240 messages per minute or 5,000 per hour to one device is throttled. Batch chat bursts into one summary.
Project quota exceeded	HTTP 429. Retry with exponential backoff and jitter; spread large campaigns over several minutes.

10.3 Taps and navigation

Case	Handling
Tap launches the app from terminated	`getInitialTap()` in a post-frame callback, after the router exists. Done in `lib/main.dart`.
Same launch tap handled twice	Prevented: `getInitialTap()` returns a value at most once (`_initialTapConsumed`).
Tap arrives before the navigator is ready	The listener checks `rootNavigatorKey.currentContext`. Terminated-state taps go through `getInitialTap()` instead.
Unknown or missing route	Missing route: tap is logged, no navigation. Unknown route: `errorBuilder` shows home.
Route needs login	Add a go_router `redirect` that sends to login and keeps the target location to resume after sign-in.
Notification for an account no longer signed in	Include `account_id` in data; ignore or show a message if it does not match the current user.
Malformed local-notification payload	`_decodePayload` catches the error and returns null; no crash.

10.4 Configuration and operations

- **Channel deleted or muted by the user**: Android recreates it with the user's settings; nothing can override a muted channel.
- **Firebase not configured**: `main()` catches the initialization error so the app still starts, and the home screen shows it as not connected.
- **Package name or bundle ID changed**: re-run `flutterfire configure`; old tokens belong to the old app and stop working.

- **Service account key leaked**: anyone can send to all your users. Disable the key in Google Cloud IAM immediately; prefer keyless workload identity.
- **Time zones and quiet hours**: schedule campaigns per user time zone (store `timezone` on the device row) and avoid sending at night.

11. FCM pricing, quotas and cost estimates

11.1 Is FCM free at large scale?

Yes. Firebase Cloud Messaging is a no-cost product on both the Spark (free) and Blaze (pay as you go) plans. There is no charge per message, per device or per user, and no paid tier of FCM itself. APNs, which FCM uses for iOS, is also free; the only Apple cost is the Apple Developer Program membership (USD 99 per year), which you need anyway to publish.

What FCM does have is **rate limits**, not prices. The main one is the per-project downstream quota of 600,000 messages per minute on the HTTP v1 API. Google states this covers over 99% of FCM developers. Increases of up to 25% can be requested when usage is regularly at least 80% of quota and the client error rate is under 5%. Requests above 18 million messages per minute need at least 30 days' notice.

Where the money goes: the cost of a push system is the backend around FCM, not FCM. That means the servers or Cloud Functions that send messages, the database that stores tokens, queues, logging and BigQuery export of delivery data. These follow normal Google Cloud or other hosting prices.

11.2 Cost estimate for several large apps

The scenario below is an illustration. Real costs depend on your region, hosting provider and send pattern, so check current prices before budgeting.

Assumption	Value
Apps	3 apps, each in its own Firebase project
Users	1,000,000 monthly active users per app (3 M total)
Devices	1.5 devices per user on average, so 4.5 M tokens
Personal messages	3 per user per day, so about 270 M per month
Broadcasts	1 topic message per app per day

Cost item	Estimate per month (USD)	Notes
FCM messages (all 270 M + broadcasts)	**0**	No charge at any volume.
APNs	**0**	Apple Developer membership is about 8 per month, needed anyway.
Token database reads	50 to 300	One device lookup per personal message. Firestore at about 0.03 to 0.06 per 100k reads is 80 to 160; a managed Postgres instance with indexes is a similar flat amount.
Token database writes	5 to 20	Logins, refreshes and daily last-seen updates.
Send workers (Cloud Run, Functions or a VM)	50 to 300	About 100 requests per second on average with campaign peaks; a few small always-on instances.
Queue (Cloud Tasks / Pub/Sub)	10 to 100	Depends on whether each message or each batch is a task.
Logging and monitoring	20 to 200	Sample success logs; keep all error logs.
Total	**about 150 to 900**	For 3 M MAU and 270 M messages a month.

11.3 Quota check for this scenario

- Each app has its own 600k per minute quota, so a broadcast to all 1.5 M devices of one app via a topic is a single API call, and FCM fans it out.
- If the same broadcast were sent token by token, it would need about 2.5 minutes at full quota. Throttle campaigns to roughly 70% of quota to leave room for transactional messages.
- Topic subscribe and unsubscribe calls have their own lower limits (about 3,000 per second per project). Subscribe on the device, not by bulk server calls during a campaign.

11.4 Ways to keep costs low

- Use topics for broadcasts. They cost one call, regardless of audience.
- Cache each user's device list for a few minutes (Redis or in memory) to cut database reads during bursts.
- Use `sendEachForMulticast` (up to 500 tokens per call) to reduce worker time.
- Delete dead tokens on every send and with the daily cleanup job.
- Keep the default sampling for BigQuery export and Cloud Logging; full logs at this volume are the most common surprise bill.

11.5 Can one user receive unlimited notifications?

There is no per-user or per-message charge and no fixed per-user sending cap. One user with two Android phones and an iPhone has three tokens, and you can send to all three as often as your product needs. "Free" does not mean "technically unlimited", though: FCM and APNs apply rate, payload and storage limits (section 11.7), and users mute or uninstall apps that notify too often.

11.6 Pricing by Firebase service

Firebase service	Pricing	Free allowance and notes
Cloud Messaging (FCM)	**Free**	Every plan, any volume.
FCM device tokens	**Free**	No charge per token or device.
FCM topics	**Free**	Subscriptions per topic are unlimited.
Console notification composer	**Free**	Campaigns and test sends.
Authentication	Free for most providers	Email, Google, Apple, anonymous and other non-phone providers are free. Phone (SMS) sign-in is billed per SMS. If you upgrade to Identity Platform, 50,000 MAU are free, then billed.
Cloud Firestore	Free quota, then pay as you go	Per day: 50,000 reads, 20,000 writes, 20,000 deletes. 1 GiB stored, 10 GiB egress per month.
Cloud Functions	Free quota, then pay as you go	Needs the **Blaze** plan to deploy. Per month: 2 million invocations, 400,000 GB-seconds, 200,000 CPU-seconds, 5 GB internet egress.

Example: 10,000 users, 3 notifications each per day, one Cloud Function call and one Firestore device lookup per notification. That is about 900,000 invocations a month (under 2 million) and 30,000 reads a day (under 50,000), so it costs **0** on the Blaze plan. Beyond the free allowance, see the estimate in section 11.2.

11.7 Technical limits (free, but not unlimited)

Limit	Documented value
Project sending quota (HTTP v1)	600,000 messages per minute
Messages to one Android device	240 per minute, 5,000 per hour
Topics per app instance	2,000
Subscriptions per topic	Unlimited
Payload size	4 KB
Pending messages per offline device	100, then all are dropped and the app is told
iOS	Also subject to APNs throttling; very frequent pushes, especially silent ones, are delayed or dropped by Apple.

11.8 Bottom line and recommendation

- **You do not pay Firebase for the number of FCM notifications.** You pay only for billable services you add around it, such as Firestore, Cloud Functions or your own servers.
- **For the POC and testing:** use FCM topics. Subscribe several devices to the same topic (the demo subscribes to `all`) and send from the Firebase console.
- **For production:** store user-to-device token mappings (Firestore or your database), send from Cloud Functions or your backend with the Admin SDK to every active device of the user, and protect sending with authentication and server-side authorization (section 13).

Official references: firebase.google.com/docs/cloud-messaging (FCM documentation); firebase.google.com/pricing (pricing plans and calculator); firebase.google.com/docs/cloud-messaging/throttling-and-quotas.

12. Testing, OS versions, test scenarios and go-live checklist

12.1 Test matrix (real devices)

Test	Android	iOS
Permission prompt, allow and deny	Android 13+	Yes
Token shown, copied, and send from console works	Yes	Yes (real iPhone)
Foreground message shows a banner, no duplicate	Local notification	Native banner
Background message, tap opens the route	Yes	Yes
Terminated, tap opens the route once	Yes	Yes
Data-only message runs the background handler	Yes	Not after force-quit
Topic subscribe/unsubscribe, topic send	Yes	Yes
Logout then send to old token gives UNREGISTERED	Yes	Yes
Account switch: no messages for previous user	Yes	Yes
Release build (tree-shaking, signing)	Yes	TestFlight (production APNs)

Quick send without a backend: Firebase console → Messaging → New campaign → Send test message, paste the token from the home screen, and add the custom data `route = /details`, `id = 42`.

12.2 Supported OS versions

Platform / version	What differs	What to test
Android 5.0 to 7.1 (API 21 to 25)	No notification channels; permission granted at install.	Notification shows with default importance.
Android 8 to 12 (API 26 to 32)	Channels required; user can mute a channel.	Heads-up banner on `high_importance_channel`.
Android 13+ (API 33+)	Runtime `POST_NOTIFICATIONS` permission; app starts with notifications off.	Allow, deny, and "Don't ask again" flows.
Android 14/15	Stricter background limits and full-screen intent rules.	Background and terminated delivery on a recent Pixel and on Samsung/Xiaomi.
iOS 12 to 14	Provisional authorization available.	Quiet delivery to Notification Centre.
iOS 15+	Focus modes and notification summary can delay or hide banners.	Delivery with Focus on; `interruption-level` if needed.
iOS 16.4+ / 17 / 18	Same APNs flow; simulator receives pushes on Apple-silicon Macs, but real devices are the reference.	Real iPhone, debug and TestFlight builds.

Minimums for this project: Android `minSdk 21` (Flutter default) and the iOS deployment target set in `ios/Podfile`. Check both against the current firebase_messaging release notes before upgrading the plugins.

12.3 Go-live checklist

- Set `enableLogs: false` (logs are already off in release builds).
- Replace the notification small icon with a monochrome drawable.
- Move the permission prompt after an explanation screen (`requestPermissionOnInit: false`).
- Add `PushTokenSync` and the device registry (sections 7 and 8).
- Implement logout cleanup and the account-switch rules (section 9).
- Confirm the APNs .p8 key is uploaded and the release build has the production `aps-environment`.
- Store the service account credential in a secret manager, or use workload identity.
- Add alerts for `THIRD_PARTY_AUTH_ERROR`, a spike in `UNREGISTERED`, and 429 rates.
- Mention push notifications in the privacy policy and the Play Data safety and App Store privacy forms.

12.4 Test scenarios and execution record

Run each scenario on at least one Android device and one real iPhone. Record the device, OS version, build type and result in the last two columns. Results are filled in by the tester; this document does not claim results that were not recorded.

ID	Scenario	Expected result	Android	iOS
PN-01	Fresh install, first launch	Permission prompt (Android 13+, iOS); token shown on home		
PN-02	Deny permission	App works; home shows "Denied" and a settings link		
PN-03	Send to token, app in foreground	One banner, no duplicate; appears in log		

ID	Scenario	Expected result	Android	iOS
PN-04	Send to token, app in background	System notification; tap opens route		
PN-05	Send to token, app killed	System notification; tap opens route once		
PN-06	Payload route /details?id=42	Details screen shows id = 42		
PN-07	Payload route /profile?user=7	Profile screen shows user = 7		
PN-08	Payload with unknown route	Home screen, no crash		
PN-09	Payload without route	App opens, no navigation		
PN-10	Data-only message, background	Background handler log entry		
PN-11	Topic "all" send	Received on every subscribed device		
PN-12	Delete token / reinstall	New token; old token returns UNREGISTERED		
PN-13	Token refresh	`onTokenRefresh` fires; server row updated		
PN-14	Logout, then send to old token	No notification for logged-out user		
PN-15	Same user on two devices	Both devices receive		
PN-16	Account switch on one device	Only the new account's messages arrive		
PN-17	Route needing login, signed out	Login, then target screen		
PN-18	Expired link	Home with expiry message		
PN-19	Airplane mode, send, reconnect	Delivered after reconnect within TTL		
PN-20	Release / TestFlight build	Same results as debug (production APNs)		
PN-21	Action button "Track", app killed	Opens /orders/42/track; back goes to order 42		
PN-22	Action button "Mark read" (no UI)	App stays closed; API called; notification removed		
PN-23	Body tap vs button tap	Body opens `route`, button opens `route_<action>`; no double navigation		
PN-24	Payload with `stack`	All screens pushed in order; back walks the stack		
PN-25	Private message sent by backend	Only the signed-in user's devices receive		

13. Hybrid delivery model: topics and backend-managed sends

This is the industry-standard approach at scale. Large apps use both delivery styles, chosen per notification type.

Use	Delivery	Examples	Why
Public / global broadcasts	**Client topics**: the app calls `subscribeToTopic`.	`all`, `news`, `offers`, `news_en`	One API call reaches millions; no token storage needed.
Private / sensitive	**Backend-managed**: tokens stored per user, sent with the Admin SDK.	Order updates, payment receipts, private messages, security alerts	Only that user's devices receive it; the server decides who is authorized.

Security rule: topics are not private. Any install of your app can subscribe to any topic name, and a topic cannot be limited to one user. Never put personal data in a topic message, and never use a topic like `user_123` for private notifications.

13.1 Backend-managed flow

1. User signs in. The app gets its FCM token and calls `POST /api/save-fcm-token` with the token, platform and installation ID.
2. The backend reads the **user ID from the verified auth token** (for example a Firebase ID token or your JWT), not from the request body. Otherwise anyone could register a token against another user.
3. The backend upserts the device row (section 7.2).
4. When an event happens (order shipped, payment received), the backend looks up the user's active devices and sends with the Firebase Admin SDK (Node.js, Python, Java or Go).
5. On token refresh the app calls the same endpoint again. On logout it calls `DELETE /api/fcm-token` and then `deleteToken()`.

13.2 Flutter client

```
// lib/services/push_token_sync.dart
Future<void> saveFcmToken(String idToken) async {
  final token = await FcmPushKit.instance.getToken();
  if (token == null) return;                       // permission denied or no APNs yet
  await http.post(
    Uri.parse('$apiBase/api/save-fcm-token'),
    headers: {'Authorization': 'Bearer $idToken', 'Content-Type': 'application/json'},
    body: jsonEncode({
      'fcmToken': token,
      'installId': await installId(),              // UUID kept in secure storage
      'platform': Platform.isIOS ? 'ios' : 'android',
    }),
  );
}

// Call after login, and on every refresh:
FcmPushKit.instance.onTokenRefresh.listen((_) => saveFcmToken(currentIdToken));
```

13.3 Backend endpoint and send (Node.js, Express)

```
import { getAuth } from 'firebase-admin/auth';
import { getMessaging } from 'firebase-admin/messaging';

app.post('/api/save-fcm-token', async (req, res) => {
  const idToken = req.headers.authorization?.replace('Bearer ', '');
  const { uid } = await getAuth().verifyIdToken(idToken);   // user ID from auth, not body
  const { fcmToken, installId, platform } = req.body;
  await db.devices.upsert({
    where:  { installId },
    create: { installId, userId: uid, fcmToken, platform, lastSeenAt: new Date() },
    update: { userId: uid, fcmToken, lastSeenAt: new Date() },
  });
  res.sendStatus(204);
});

// Private notification: called by your order service
export async function notifyOrderShipped(userId, orderId) {
  const devices = await db.devices.findMany({ where: { userId } });
  await getMessaging().sendEachForMulticast({
    tokens: devices.map(d => d.fcmToken),
    notification: { title: 'Order shipped', body: `Order #${orderId} is on its way` },
    data: { route: `/orders/${orderId}` },
  });                                   // then delete UNREGISTERED tokens (section 7.3)
}

// Public broadcast: one call
await getMessaging().send({ topic: 'offers',
  notification: { title: 'Weekend sale', body: '20% off' }, data: { route: '/offers' } });
```

13.4 Same flow with Cloud Functions and Firestore

```
// functions/index.js  (Blaze plan)
import { onDocumentUpdated } from 'firebase-functions/v2/firestore';
import { getFirestore } from 'firebase-admin/firestore';
import { getMessaging } from 'firebase-admin/messaging';

export const orderShipped = onDocumentUpdated('orders/{orderId}', async (event) => {
  const before = event.data.before.data(), after = event.data.after.data();
  if (before.status === after.status || after.status !== 'shipped') return;

  const snap = await getFirestore()
      .collection(`users/${after.userId}/devices`).get();   // written by the app on login
  const tokens = snap.docs.map(d => d.get('fcmToken'));
  if (tokens.length === 0) return;
  await getMessaging().sendEachForMulticast({
    tokens,
    notification: { title: 'Order shipped', body: `Order #${event.params.orderId}` },
    data: { route: `/orders/${event.params.orderId}` },
  });
});
```

Protect `users/{uid}/devices` with Firestore security rules so a user can only write their own devices: `allow write: if request.auth.uid == uid;`

13.5 Python Admin SDK equivalent

```
import firebase_admin
from firebase_admin import messaging
firebase_admin.initialize_app()

msg = messaging.MulticastMessage(
    tokens=tokens,
    notification=messaging.Notification(title='Payment received', body='Receipt #881'),
    data={'route': '/payments/881'},
)
resp = messaging.send_each_for_multicast(msg)
```

14. Notification action buttons

Action buttons, for example **Track**, **Pay now**, **Mark as read** or **Reply**, are possible on both platforms. Each button can open a different screen or run a short task without opening the app. The two platforms build them differently, which decides the payload.

	Android	iOS
Where buttons come from	The app builds the notification with `flutter_local_notifications` and `AndroidNotificationAction`.	A **category** registered at app launch. The push names it with `aps.category`; iOS draws the buttons.
Payload type	**Data-only**. A message with a `notification` block is drawn by the system in the background, without buttons.	**Alert** with `category` in `apns.payload.aps`. Works even when the app was force-quit.
Maximum buttons	3	4 (2 visible in the banner, more on long press)
Button that opens the app	`showsUserInterface: true`	Action option `foreground`
Button that does not open the app	Runs `onDidReceiveBackgroundNotificationResponse` in a background isolate.	App gets a few seconds in the background.
Inline text reply	`AndroidNotificationActionInput`	Text input action
Reading which button was tapped	`NotificationResponse.actionId`	Native `UNNotificationResponse.actionIdentifier`; needs a small bridge (14.4).

14.1 One payload for both platforms

Leave out the top-level `notification` block. Android then gets data only and the app draws the notification with buttons; iOS gets the alert from the `apns` block. Routes per button use the key `route_<actionId>`.

```
{
  "message": {
    "token": "<FCM token>",
    "data": {
      "title": "Order #42 shipped",
      "body": "Arriving tomorrow",
      "category": "ORDER_ACTIONS",
      "route": "/orders/42",                 // tap on the notification body
      "route_track": "/orders/42/track",     // tap on "Track"
      "route_invoice": "/orders/42/invoice"  // tap on "Invoice"
    },
    "android": { "priority": "HIGH" },
    "apns": {
      "headers": { "apns-priority": "10" },
      "payload": { "aps": {
        "alert": { "title": "Order #42 shipped", "body": "Arriving tomorrow" },
        "category": "ORDER_ACTIONS",
        "sound": "default"
      } }
    }
  }
}
```

14.2 Declare the categories once in the app

```
// Same ids on both platforms; titles can be localized in the app.
const orderActions = [
  ('track', 'Track', true),         // (id, title, opensApp)
  ('invoice', 'Invoice', true),
  ('mark_read', 'Mark as read', false),
];

// iOS: register at init (in fcm_push_kit _initLocalNotifications)
DarwinInitializationSettings(
  notificationCategories: [
    DarwinNotificationCategory('ORDER_ACTIONS', actions: [
      for (final (id, title, opens) in orderActions)
        DarwinNotificationAction.plain(id, title, options: {
          if (opens) DarwinNotificationActionOption.foreground,
        }),
    ]),
  ],
)

// Android: attach when showing a data-only message
AndroidNotificationDetails(channelId, channelName,
  importance: Importance.max, priority: Priority.high,
  actions: [
    for (final (id, title, opens) in orderActions)
      AndroidNotificationAction(id, title,
          showsUserInterface: opens, cancelNotification: true),
  ],
)
```

14.3 Android: show data-only messages and read the button

```
// lib/main.dart: background and killed state
@pragma('vm:entry-point')
Future<void> backgroundMessageHandler(RemoteMessage message) async {
  await fcmPushKitDefaultBackgroundHandler(message);
  if (message.notification == null && message.data['title'] != null) {
    // New isolate: initialize local notifications here before show().
    await FcmPushKit.instance.showFromData(message.data);
  }
}

// fcm_push_kit: pass the button id into the tap
onDidReceiveNotificationResponse: (r) {
  final data = _decodePayload(r.payload ?? '');
  if (data != null) _emitTap(data, TapSource.foreground, null, actionId: r.actionId);
},
onDidReceiveBackgroundNotificationResponse: onActionInBackground,  // top-level function

@pragma('vm:entry-point')
void onActionInBackground(NotificationResponse r) {
  // "Mark as read": short API call only, no UI, no router.
}
```

Two changes are needed in the current kit: `_onForegroundMessage` returns early when `notification` is null, so it must also show data-only messages; and `getInitialTap()` should read `launch.notificationResponse.actionId` for a button that launched the app from killed state.

14.4 iOS: forward the button id from AppDelegate

`firebase_messaging` reports the tap but not which button was pressed, and `flutter_local_notifications` only reports buttons on notifications it created itself. A small override in `ios/Runner/AppDelegate.swift` forwards the button id to Dart and then calls `super` so both plugins keep working.

```
override func userNotificationCenter(_ center: UNUserNotificationCenter,
                                     didReceive response: UNNotificationResponse,
                                     withCompletionHandler completionHandler: @escaping () -> Void) {
  let id = response.actionIdentifier
  if id != UNNotificationDefaultActionIdentifier && id != UNNotificationDismissActionIdentifier {
    var data = response.notification.request.content.userInfo
    data.removeValue(forKey: "aps")
    // MethodChannel "fcm_push_kit/actions"; queue it if Dart is not listening yet (cold start).
    PushActionBridge.shared.deliver(actionId: id, data: data)
  }
  super.userNotificationCenter(center, didReceive: response,
                               withCompletionHandler: completionHandler)
}
```

Test whether `onMessageOpenedApp` also fires for a button tap on your plugin version. If it does, ignore it when an action with the same `gcm.message_id` has just arrived, so the user is not navigated twice. Alternative: the `awesome_notifications` and `awesome_notifications_fcm` plugins handle buttons on both platforms, but they replace `flutter_local_notifications`.

14.5 Map the button to a route

```
// push_notification_tap.dart
final String? actionId;          // null = tap on the body

String? get route {
  final value = (actionId != null ? data['route_$actionId'] : null) ?? data[routeKey];
  return value is String && value.isNotEmpty ? value : null;
}
// In `location`, skip keys starting with "route_" and "title", "body", "category",
// so they do not become query parameters.
```

14.6 Limits and caveats

- Android: data-only messages are not delivered if the user pressed **Force stop** in system settings, and some manufacturers (Xiaomi, Oppo, Vivo) delay them under battery saving. Keep `priority: HIGH` and always show a notification for it, or Android lowers the app's priority.
- iOS: the button titles come from the category in the app, not the payload. To change them, ship an app update or register several categories.
- Buttons that do not open the app must finish quickly (a few seconds) and cannot navigate.
- Route every button through the same auth and expiry checks as a normal tap (section 6.4).

15. Multi-screen navigation from one tap

Yes, this is possible. A tap can open a screen with the right screens behind it (so Back works naturally), open several screens in order, or open a screen that moves on to another after the user completes a step. Choose one of these three patterns.

15.1 Nested routes (recommended)

Declare child routes in go_router. `router.go('/orders/42/track')` then builds the whole stack: Orders, then Order 42, then Tracking. The payload stays a single `route`.

```
GoRoute(path: '/orders', builder: (c, s) => const OrdersScreen(), routes: [
  GoRoute(path: ':id', builder: (c, s) => OrderScreen(id: s.pathParameters['id']!),
    routes: [
      GoRoute(path: 'track',
          builder: (c, s) => TrackScreen(id: s.pathParameters['id']!)),
      GoRoute(path: 'invoice',
          builder: (c, s) => InvoiceScreen(id: s.pathParameters['id']!)),
    ]),
]),
// Payload: { "route": "/orders/42/track" }  ->  Back goes to Order 42, then Orders.
```

15.2 Explicit stack in the payload

For screens that are not nested, send the screens in order in a `stack` key, separated by `|`. The first replaces the current location, and the rest are pushed.

```
// Payload: { "stack": "/profile?user=7|/details?id=42" }
void openFromTap(PushNotificationTap tap) {
  final stack = (tap.data['stack'] as String?)?.split('|');
  if (stack == null || stack.isEmpty) {
    final location = tap.location;
    if (location != null) AppRouter.router.go(location);
    return;
  }
  AppRouter.router.go(stack.first);
  for (final location in stack.skip(1)) {
    AppRouter.router.push(location);
  }
}
```

This works with the demo's current flat `/details` and `/profile` routes without changing the router. Each pushed location still goes through `redirect`, so login and expiry checks apply to every step.

15.3 "Next" screen after an action

For flows such as "pay, then show the receipt", send `route` for the first screen and a `next` parameter. The screen navigates to `next` after the user completes its step, the same way login uses `next` in section 6.4.

```
// Payload: { "route": "/payments/881", "next": "/payments/881/receipt" }
onPaymentSuccess: () {
  final next = GoRouterState.of(context).uri.queryParameters['next'];
  next != null ? context.go(next) : context.pop();
}
```

15.4 Rules for all patterns

- Validate every location against the known routes. Ignore unknown entries instead of failing the whole stack.
- Limit the stack to a few screens (3 to 4); longer stacks confuse Back navigation.

- Cold start: run the navigation from `getInitialTap()` after the first frame, as the demo already does.
- Combine with action buttons: `route_track` can be a nested route such as `/orders/42/track`.

Appendix A. FCM error codes (HTTP v1)

Code	HTTP	Meaning	Action
`UNREGISTERED`	404	Token no longer valid (uninstall, logout, expiry).	Delete the token.
`INVALID_ARGUMENT`	400	Bad token format or bad payload (e.g. over 4 KB, non-string data).	Fix the payload; delete the token if the token was the problem.
`SENDER_ID_MISMATCH`	403	Token belongs to a different Firebase project.	Send with the right project's credentials; check `app_id`.
`QUOTA_EXCEEDED`	429	Project, device or topic rate exceeded.	Exponential backoff; honour `Retry-After`.
`UNAVAILABLE`	503	FCM temporarily overloaded.	Retry with backoff.
`INTERNAL`	500	Unknown FCM error.	Retry with backoff.
`THIRD_PARTY_AUTH_ERROR`	401	APNs key or certificate invalid or missing.	Fix the APNs configuration in Firebase.

Appendix B. Limits reference

Limit	Value
Downstream messages per project	600,000 per minute (HTTP v1 default)
Message payload size	4,096 bytes (data plus notification)
Tokens per multicast call	500
Messages to one device	About 240 per minute and 5,000 per hour
Pending messages stored per offline device	100 (non-collapsible)
Collapse keys per device (Android)	4 at a time
Maximum time to live	28 days (also the default)
Topics per app install	2,000
Topic management requests	About 3,000 per second per project
Android token expiry	270 days of inactivity
Background handler run time (Android)	About 20 seconds

Sources: firebase.google.com/pricing; firebase.google.com/docs/cloud-messaging/throttling-and-quotas; firebase.google.com/docs/cloud-messaging/manage-tokens. Limits change over time; check these pages before planning large campaigns.